Dashboards for Learning Model Monitoring

Executive Summary

This document specifies a comprehensive dashboard system for monitoring the BitX AI engine's learning and performance metrics. Building upon the existing Local Agent Dashboard, these specifications define 5 dashboard categories: Operational, Model Performance, Agent Effectiveness, Knowledge & Learning, and Quality Assurance.

The dashboards transform raw telemetry data — stored in SQLite tables (`agent_runs` , `agent_steps` , `agent_artifacts` , `rag_documents` , `rag_search_log`) — into actionable insights for engineering leadership, data science teams, and operations staff.

---

1. Dashboard Strategy

1.1 Target Audience

Audience	Primary Interest	Key Dashboards
Engineering Leadership	System health, throughput, reliability	Operational, Model Performance
Data Science	Model efficiency, learning quality, RAG effectiveness	Model Performance, Knowledge & Learning
Operations	Uptime, queue management, alerts	Operational, Quality Assurance
Product Management	Agent effectiveness, feature coverage	Agent Effectiveness

1.2 Dashboard Categories

```
+-------------------+     +-------------------+     +-------------------+
|   OPERATIONAL     |     | MODEL PERFORMANCE |     | AGENT EFFECTIVE.  |
|                   |     |                   |     |                   |
| System health     |     | Token analytics   |     | Success rates     |
| Queue monitoring  |     | Latency metrics   |     | Tool usage        |
| Activity stream   |     | Throughput        |     | Stage durations   |
+-------------------+     +-------------------+     +-------------------+

+-------------------+     +-------------------+
| KNOWLEDGE &       |     | QUALITY           |
| LEARNING          |     | ASSURANCE         |
|                   |     |                   |
| RAG index health  |     | Eval benchmarks   |
| Search effective. |     | Safety metrics    |
| Coverage analysis |     | Validation trends |
+-------------------+     +-------------------+
```

1.3 Data Sources

Data Source	Table	Key Metrics
Run history	`agent_runs`	Status, tokens, duration, profile, timestamps
Step details	`agent_steps`	Stage, model, tool name, tokens, duration
Artifacts	`agent_artifacts`	Type, file path, creation time
Approvals	`agent_approvals`	Status, reason, resolution time
RAG documents	`rag_documents`	Source type, chunk count, content hash

Search logs	`rag_search_log`	Query, result count, duration
Eval reports	JSON files	Scores, categories, pass/fail rates

---

2. Operational Dashboard

2.1 System Health Monitor

Purpose: Real-time view of LM Studio and engine health.

```
+-----------------------------------------------------+
| SYSTEM HEALTH                                       |
+-----------------------------------------------------+
|                                                     |
|  Router Model          Worker Model                 |
|  +------------------+  +------------------+         |
|  | [*] Available    |  | [*] Available    |         |
|  | ministral-8b     |  | qwen-32b-coder   |         |
|  | 16K context      |  | 8K context       |         |
|  | Temp: 0.3        |  | Temp: 0.2        |         |
|  +------------------+  +------------------+         |
|                                                     |
|  Last health check: 5s ago                          |
|  Uptime: 4h 32m                                     |
|  Status: HEALTHY                                    |
+-----------------------------------------------------+
```

Data source: `GET /local-agent/health` (polled every 10s)

2.2 Queue Monitor

Purpose: Visualize queue depth and throughput.

```
+-----------------------------------------------------+
| QUEUE STATUS                                        |
+-----------------------------------------------------+
|                                                     |
|  Active: 2/2 [========] 100%                        |
|  Queued: 3   [===-----]  6%                         |
|                                                     |
|  Queue Timeline (last 30 min):                      |
|  ▇▇▇▇▅▇▇▇▇▅▇▇▇▇▇▇▇▇▅▇▇▇▇                            |
|  (bar chart: queue depth over time)                 |
|                                                     |
|  Avg wait time: 45s                                 |
|  Max wait time: 2m 15s                              |
|  Throughput: 8 runs/hour                            |
+-----------------------------------------------------+
```

Data source: `GET /local-agent/queue` + historical run timestamps

2.3 Run Activity Stream

Purpose: Timeline of recent run activity with status distribution.

```
+-----------------------------------------------------+
| RUN ACTIVITY (Last 24 Hours)                        |
+-----------------------------------------------------+
|                                                     |
|  Status Distribution:                               |
|  Completed: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 42 (89%)            |
|  Failed:    ▓▓▓                   3  (6%)           |
|  Running:   ▓▓                    2  (4%)           |
|  Cancelled: ▓                     1  (2%)           |
|                                                     |
|  Runs/Hour (24h):                                   |
|  12am  4am  8am  12pm  4pm  8pm  Now                |
|   2    1    5    8    12    6    4                  |
|  ▂▂   ▁▁   ▅▅   ▆▆   ▇▇   ▆▆   ▃▃                  |
+-----------------------------------------------------+
```

Data source: `agent_runs` table (grouped by hour, status)

---

3. Model Performance Dashboard

3.1 Token Usage Analytics

Purpose: Understand token consumption patterns by model and profile.

```
+-----------------------------------------------------+
| TOKEN USAGE                                         |
+-----------------------------------------------------+
|                                                     |
|  Total Tokens (24h): 1,245,678                      |
|  Router: 345,678 (28%)  |  Worker: 900,000 (72%)    |
|                                                     |
|  Top 5 Profiles by Token Usage:                     |
|  product_manager     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  55,885        |
|  backend_developer   ▓▓▓▓▓▓▓▓▓▓▓▓▓    48,234        |
|  solutions_architect ▓▓▓▓▓▓▓▓▓▓▓      42,156        |
|  qa_lead             ▓▓▓▓▓▓▓▓▓▓       38,901        |
|  security_analyst    ▓▓▓▓▓▓▓▓▓        35,672        |
|                                                     |
|  Tokens per Stage:                                  |
|  intake: 12%  context: 18%  planning: 8%            |
|  tool_loop: 55%  validation: 2%  finalize: 5%       |
+-----------------------------------------------------+
```

Data source: `agent_steps.tokens_used` + `agent_steps.stage` + `agent_steps.role`

3.2 Latency Metrics

Purpose: Track response times across models and stages.

```
+-----------------------------------------------------+
| LATENCY (Last 24 Hours)                             |
+-----------------------------------------------------+
|                                                     |
|  Router Model (Ministral 8B):                       |
|  Avg: 2.3s  |  P50: 1.8s  |  P95: 5.2s  |  P99: 8.1s|
|                                                     |
```

```
|  Worker Model (Qwen 32B):                                |
|  Avg: 8.7s  |  P50: 6.4s  |  P95: 18.3s |  P99: 28.5s|
|                                                          |
|  Stage Duration Breakdown:                               |
|  intake          ██         2.3s avg                     |
|  context_gather  ═══        8.5s avg                     |
|  planning        ═══        4.1s avg                     |
|  tool_loop       ═══════════════  28.7s avg              |
|  validation      ═          0.8s avg                     |
|  finalize        ═          1.9s avg                     |
+-------------------------------------------------------+
```

Data source: `agent_steps.duration_ms` grouped by model and stage

3.3 Throughput Metrics

Purpose: Monitor system capacity and utilization.

Metric	Calculation	Target
Runs per hour	Count of completed runs / hours	> 8
Avg run duration	Mean of `total_duration_ms`	< 120s
Concurrent utilization	Time at max concurrent / total time	< 80%
Queue drain rate	Completions per minute during load	> 0.5/min

3.4 Model Comparison

```
+-------------------------------------------------------+
| MODEL COMPARISON                                      |
+-------------------------------------------------------+
|                                                        |
| Metric              | Router (8B) | Worker (32B)     |
|---------------------|-------------|------------------|
| Avg response time   | 2.3s        | 8.7s             |
| Tokens per request  | 1,245       | 4,892            |
| Success rate        | 98.2%       | 95.1%            |
| Retry rate          | 1.2%        | 3.8%             |
| Fallback triggered  | N/A         | 2.1%             |
| Context utilization | 38%         | 72%              |
+-------------------------------------------------------+
```

---

4. Agent Effectiveness Dashboard

4.1 Success Rate by Profile

Purpose: Identify which agents succeed most and which need attention.

```
+-------------------------------------------------------+
| AGENT SUCCESS RATES                                   |
+-------------------------------------------------------+
|                                                        |
| Profile               Completed  Failed  Rate         |
| business_analyst      ═══════    ═       89%          |
| process_analyst       ═══════    ═       89%          |
```

```
| backend_developer     ██████    ██      78%         |
| qa_lead               ████████  -       100%        |
| security_analyst      ██████    ██      78%         |
| product_manager       ██████████ -      100%        |
| ...                                                  |
|                                                      |
| Overall: 89% success rate (42/47 runs)               |
+-----------------------------------------------------+
```

Data source: `agent_runs` grouped by `profile_id` and `status`

4.2 Tool Usage Heatmap

Purpose: Understand which tools each agent profile uses most.

```
+-----------------------------------------------------+
| TOOL USAGE BY PROFILE                               |
+-----------------------------------------------------+
|                                                      |
|              search read  search write apply  run    |
|              files  file  content file  patch  tests |
| backend_dev  ██    ████  ███    ███   ██     ▌      |
| qa_lead      ▌     ███   ██     -     -      ██     |
| proc_analyst ██    ████  ██     ██    -      -      |
| solutions_ar ▌     ████  ███    ▌     -      -      |
|                                                      |
| (intensity = frequency of use)                       |
+-----------------------------------------------------+
```

Data source: `agent_steps.tool_name` grouped by profile via `agent_runs.profile_id`

4.3 Stage Duration Breakdown

Purpose: Identify which pipeline stages take longest per profile.

```
+-----------------------------------------------------+
| STAGE DURATION BY PROFILE (avg seconds)             |
+-----------------------------------------------------+
|                                                      |
|              intake context plan  tools  valid final |
| backend_dev   3.2   12.5   4.8   35.2   1.2   2.1  |
| qa_lead       2.1    8.3   3.5   22.1   0.8   1.8  |
| proc_analyst  2.5    9.7   5.2   28.4   0.6   1.5  |
| PM            4.8   15.2   8.3   42.1   1.5   3.2  |
+-----------------------------------------------------+
```

Data source: `agent_steps.duration_ms` grouped by `stage` and `run.profile_id`

4.4 Validation Results

Purpose: Track common validation failures.

Validator	Pass Rate	Common Failures
`blocked_file_patterns`	100%	No violations
`tool_restrictions`	98%	Write tool used by router profile

`allowed_paths`	97%	Path outside workspace boundary
`diff_threshold`	95%	Large diff from backend_developer
`output_schema`	92%	Missing filePath on artifacts
`approval_required`	100%	(warnings only)
`required_tests`	88%	Test suites not configured

Data source: Validation report artifacts from `agent_artifacts` where `type = 'report'`

4.5 Approval Metrics

Metric	Value	Trend
Approval requests (30d)	12	Stable
Approval rate	92%	Up
Avg response time	4.2 hours	Down
Auto-approved (no risk)	78%	Up

Data source: `agent_approvals` table

---

5. Knowledge & Learning Dashboard

5.1 RAG Index Health

Purpose: Monitor the knowledge base size and freshness.

```
+-----------------------------------------------------+
| RAG INDEX HEALTH                                    |
+-----------------------------------------------------+
|                                                     |
|  Total Documents: 342 chunks                        |
|  Last Ingestion: 2 hours ago                        |
|                                                     |
|  By Source Type:                                    |
|  doc          ################  156 (46%)          |
|  profile      ########           78 (23%)           |
|  skill        ######             52 (15%)           |
|  schema       ####               36 (11%)           |
|  process      ##                 12 (4%)            |
|  test_history #                   5 (1%)            |
|  story        #                   3 (1%)            |
+-----------------------------------------------------+
```

Data source: `rag_documents` grouped by `source_type`

5.2 Search Effectiveness

Purpose: Understand how well RAG search serves agent needs.

```
+-----------------------------------------------------+
| SEARCH EFFECTIVENESS (Last 7 Days)                  |
+-----------------------------------------------------+
```

```
|                                                      |
|  Total Searches: 234                                 |
|  Avg Results: 3.8 per search                         |
|  Zero-Result Rate: 12%                               |
|  Avg Search Duration: 45ms                           |
|                                                      |
|  Result Count Distribution:                          |
|  0 results  ████        12%                          |
|  1-2 results ██████     28%                          |
|  3-5 results ################  52%                  |
|  5+ results  ██          8%                          |
|                                                      |
|  Zero-Result Queries (needs attention):              |
|  - "kubernetes deployment" (searched 5x)             |
|  - "payment processing" (searched 3x)                |
|  - "notification service" (searched 2x)              |
+-----------------------------------------------------+
```

Data source: `rag_search_log` table

5.3 Context Quality Indicators

Metric	Value	Target	Status
Avg top result score	0.72	> 0.5	Good
Searches with score > 0.5	78%	> 70%	Good
Avg results per search	3.8	3-5	Good
Zero-result rate	12%	< 10%	Attention
Avg search latency	45ms	< 100ms	Good

5.4 Knowledge Coverage Map

Purpose: Identify gaps in knowledge coverage.

```
+-----------------------------------------------------+
| KNOWLEDGE COVERAGE                                  |
+-----------------------------------------------------+
|                                                      |
| Domain              | Docs | Searches | Gap Score    |
|---------------------|------|----------|--------------|
| Authentication      | 12   | 45       | Low          |
| Tenant Management   | 8    | 32       | Low          |
| User Management     | 6    | 28       | Low          |
| Pipeline Execution  | 4    | 22       | Medium       |
| Notifications       | 1    | 18       | HIGH         |
| Payment Processing  | 0    | 12       | CRITICAL     |
| Kubernetes Deploy   | 0    | 8        | HIGH         |
|                                                      |
| Gap Score = Searches / (Docs + 1)                    |
| CRITICAL = high searches, zero docs                  |
+-----------------------------------------------------+
```

---

6. Quality Assurance Dashboard

6.1 Eval Benchmark Trends

Purpose: Track quality over time across benchmark runs.

```
+-----------------------------------------------------+
| BENCHMARK TRENDS                                     |
+-----------------------------------------------------+
|                                                      |
|  Last Benchmark: 2024-03-04                          |
|  Overall Score: 78.5%                                |
|  Pass Rate: 9/12 (75%)                               |
|                                                      |
|  Score Trend (last 5 runs):                          |
|  Run 1: 65% --> Run 2: 71% --> Run 3: 75%           |
|  --> Run 4: 78% --> Run 5: 78.5%                     |
|  (improving trend)                                   |
+-----------------------------------------------------+
```

Data source: Benchmark JSON reports in `agents-local/eval-results/`

6.2 Category Performance

```
+-----------------------------------------------------+
| EVAL CATEGORY SCORES                                 |
+-----------------------------------------------------+
|                                                      |
|  Category        Cases  Passed  Score                |
|  BA              2      2       85%  ##########     |
|  Engineering     2      1       72%  ##########     |
|  Testing         2      2       81%  ##########     |
|  Repair          1      1       76%  ##########     |
|  Adversarial     4      3       90%  ##########     |
|                                                      |
|  Strongest: Adversarial (safety)                     |
|  Weakest:   Engineering (code generation)            |
+-----------------------------------------------------+
```

6.3 Safety Metrics

Purpose: Monitor security and safety compliance.

Metric	Value	Target	Status
Blocked .env access attempts	100% blocked	100%	Pass
Blocked path traversal	100% blocked	100%	Pass
Blocked SQL injection	100% blocked	100%	Pass
Read-only profile enforcement	98% compliant	100%	Attention
Forbidden tool rejections	100% blocked	100%	Pass

---

7. Dashboard Data Architecture

7.1 Existing Data Sources (SQL)

```sql
-- Runs per profile with success rate
SELECT profile_id,
       COUNT(*) as total,
       SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) as completed,
       AVG(total_tokens) as avg_tokens,
       AVG(total_duration_ms) as avg_duration
FROM agent_runs
WHERE created_at > datetime('now', '-7 days')
GROUP BY profile_id;

-- Token usage by stage
SELECT s.stage, s.role,
       SUM(s.tokens_used) as total_tokens,
       AVG(s.duration_ms) as avg_duration
FROM agent_steps s
GROUP BY s.stage, s.role;

-- RAG search effectiveness
SELECT DATE(created_at) as day,
       COUNT(*) as searches,
       AVG(result_count) as avg_results,
       SUM(CASE WHEN result_count = 0 THEN 1 ELSE 0 END) as zero_results,
       AVG(duration_ms) as avg_duration
FROM rag_search_log
GROUP BY DATE(created_at);

-- Tool usage by profile
SELECT r.profile_id, s.tool_name, COUNT(*) as usage_count
FROM agent_steps s
JOIN agent_runs r ON s.run_id = r.id
WHERE s.tool_name IS NOT NULL
GROUP BY r.profile_id, s.tool_name;
```

7.2 New API Endpoints Required

Endpoint	Method	Response
`/local-agent/analytics/runs`	GET	Run counts by profile, status, day
`/local-agent/analytics/tokens`	GET	Token usage by model, stage, profile
`/local-agent/analytics/latency`	GET	Duration percentiles by model, stage
`/local-agent/analytics/rag`	GET	Search stats, coverage, zero-result queries
`/local-agent/analytics/tools`	GET	Tool usage by profile, success rates
`/local-agent/analytics/eval`	GET	Latest benchmark report + trend data

7.3 Aggregation Strategy

Time Window	Granularity	Use
Last 1 hour	Per-minute	Real-time operational monitoring

Last 24 hours	Per-hour	Daily performance review
Last 7 days	Per-day	Weekly trend analysis
Last 30 days	Per-day	Monthly reporting

7.4 Caching Strategy

Data	Cache Duration	Invalidation
Run analytics	60 seconds	On new run completion
Token analytics	5 minutes	Time-based
RAG stats	10 minutes	On new ingestion
Eval results	Until new benchmark	On benchmark completion

---

8. Implementation Specifications

8.1 Dashboard Route Structure

```
/local-agents/analytics
  ├── /operational      Operational Dashboard
  ├── /performance      Model Performance Dashboard
  ├── /agents           Agent Effectiveness Dashboard
  ├── /knowledge        Knowledge & Learning Dashboard
  └── /quality          Quality Assurance Dashboard
```

8.2 Component Architecture

```
AnalyticsDashboard (tab container)
  ├── OperationalTab
  │     ├── HealthMonitor
  │     ├── QueueChart
  │     └── ActivityTimeline
  ├── PerformanceTab
  │     ├── TokenUsageChart
  │     ├── LatencyChart
  │     └── ModelComparisonTable
  ├── AgentsTab
  │     ├── SuccessRateChart
  │     ├── ToolUsageHeatmap
  │     └── StageDurationChart
  ├── KnowledgeTab
  │     ├── IndexHealthCard
  │     ├── SearchEffectivenessChart
  │     └── CoverageMap
  └── QualityTab
        ├── BenchmarkTrendChart
        ├── CategoryRadar
        └── SafetyMetrics
```

8.3 Chart Library Recommendation

Recharts (React-native charting):

- Lightweight, composable React components
- Built-in responsive container
- Supports: bar, line, area, pie, radar, heatmap
- Dark mode compatible with Tailwind colors

8.4 Data Refresh Strategy

Dashboard Tab	Refresh Interval	Fetch Strategy
Operational	10 seconds	Polling
Performance	60 seconds	Polling
Agents	60 seconds	Polling
Knowledge	5 minutes	Polling
Quality	On-demand	Manual refresh

---

9. Alert and Notification Rules

9.1 Defined Alerts

Alert	Trigger	Severity	Action
Model Offline	Health check shows model unavailable	Critical	Toast notification + status badge change
Queue Full	Queue length >= 45 (90% of max 50)	Warning	Toast notification
High Failure Rate	> 20% failures in last hour	Warning	Dashboard banner
RAG Index Stale	No ingestion in 24+ hours	Info	Dashboard indicator
Long Queue Wait	Any run waiting > 5 minutes	Warning	Dashboard indicator
Eval Regression	Score drops > 10% from previous benchmark	Warning	Dashboard banner

9.2 Alert Display

Alerts appear as:

- **Critical**: Red banner at top of dashboard, persistent until resolved
- **Warning**: Orange toast notification, auto-dismiss after 30 seconds
- **Info**: Blue indicator in relevant dashboard section

Appendix A: Wireframe Key

Symbol	Meaning
`▇▇▇`	Filled bar (proportional to value)
`▓▓▓`	Partially filled bar
`[*]`	Status indicator (filled = available)
`[ ]`	Status indicator (empty = unavailable)
`[Cancel]`	Action button
`-->`	Trend direction

Appendix B: Metric Definitions

Metric	Formula
Success Rate	completed_runs / (completed_runs + failed_runs) * 100
Throughput	completed_runs / time_period_hours
Token Efficiency	useful_output_tokens / total_tokens
Search Effectiveness	searches_with_results / total_searches
Gap Score	search_count / (document_count + 1)
Context Utilization	avg_prompt_tokens / context_window_size

Cross-references: AI Engine Architecture (Section 7: Run Persistence) | Learning Methodology (Section 8: Continuous Improvement) | UI/UX Design (Section 5: Local Agent Dashboard)